

FA26: DATA-266 Sec 21 & 71 - Generative AI and LLM

Homework 6

Self-Supervised and Contrastive Representation Learning (STL-10)

Siva Surya Chandran | 019130215

SID4	SEED	SLICE	HP_ID	CLS_A	CLS_B
3215	3215	215	5	5	2

Note on SID4. My SJSU ID is 019130215, so the last four digits are 0215. Taken literally that has a leading zero, which collapses to the 3-digit number 215 when used as a number and makes SID4 ambiguous with SLICE. I used 3215 instead - the last four digits with the preceding digit 3 in place of the leading zero - so that SID4 is a genuine 4-digit value and SEED/SLICE stay distinct. All parameters are derived from SID4 = 3215.

HW6 has no HP_ID mapping and no data slice, so HP_ID and SLICE are not used. SEED = 3215 is used for every random seed. CLS_A = 5 (dog) and CLS_B = 2 (car) are the classes of the first two query images in Part D.

What I did

I compared three ways of learning image features on STL-10 when only 500 labeled images are available: training a ResNet-18 directly on the 500 labels (Part A), pretraining it with a rotation-prediction task on unlabeled images (Part B), and pretraining it with SimCLR-style contrastive learning (Part C). For B and C I then froze the encoder and trained only a linear classifier on the same 500 labels, and in Part D I compared the three encoders by retrieving nearest neighbours of test images. Everything below - the code, all the outputs and the analysis - is in the notebook HW6.ipynb, which I ran top to bottom on my Mac (MPS GPU, about 66 minutes) with SEED = 3215.

ResNet-18 with no pretrained weights is the backbone in every part. The final test accuracy for every model is measured on all 8,000 STL-10 test images. Besides the three required models I added three things, each marked as extra below: a random untrained encoder with the same linear probe (to show how much comes from the architecture alone), five different 500-image subsets to see how much the numbers move, and a rotation model trained on the same 20k images SimCLR uses (a data-matched comparison).

Setup

STL-10 has 5,000 labeled training images, 8,000 test images and 100,000 unlabeled images, all 96x96. The 500-image labeled subset is 10% of the training split, stratified (50 per class) and drawn with SEED. The same 500 images are used in Parts A, B and C. I kept the images at 96x96 and did all augmentations as batched GPU operations, which made training fast enough on a laptop.

The SimCLR augmentations come from the course demo (Demo_6_Self_Supervised_Learning.ipynb): random resized crop (scale 0.6-1.0), horizontal flip, colour jitter (0.4 brightness, contrast and saturation, 0.1 hue) and random grayscale (p=0.2). The demo applies them to 32x32 CIFAR images with torchvision; I re-implemented them on 96x96 tensors (the hue shift is a rotation in YIQ colour space instead of HSV, which is close but not identical).



Figure 1: original images, two SimCLR views (demo augmentations), and the lighter crop + flip used for Parts A and B.

Part A - Supervised with 10% labels

I trained ResNet-18 plus a 10-way linear head end to end on the 500 images for 15 epochs (AdamW, learning rate $1e-3$ with a one-cycle schedule, batch 64, random crop and flip). Test accuracy at the end of training is 43.95%. The training loss went from 2.25 to 0.53 while the test accuracy rose to about 44% and stayed there, and during the first epochs it was very unstable (28% at epoch 3, 14% at epoch 4, 38% at epoch 6). With 500 images and 11 million parameters the network memorizes the training set. I report the last epoch, not the best one, since picking the best epoch would mean selecting on the test set.

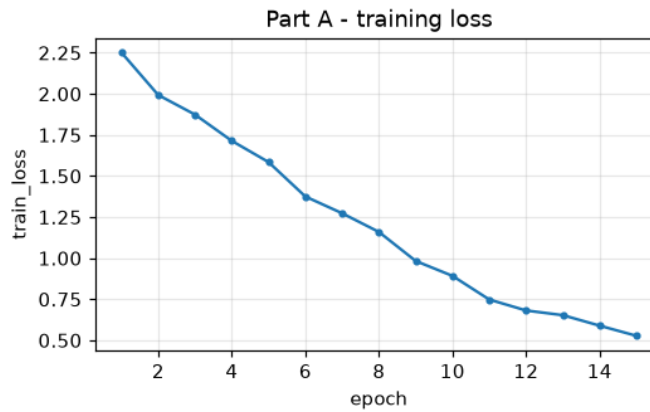


Figure 2: Part A training loss.

Part B - Rotation self-supervised learning

Every one of the 100,000 unlabeled images is given a random rotation (0, 90, 180 or 270 degrees) each time it is drawn, and a ResNet-18 plus a 4-way head predicts which one. I trained for 15 epochs (AdamW 1e-3, cosine schedule, batch 256). The rotation accuracy on training batches rose from 56% to 83.7%, and on the test images the finished model gets 89.2%, so the pretext task was learned well. I then removed the rotation head, froze the encoder (evaluation mode, no gradients, and I check an MD5 hash of the weights before and after to prove nothing changed) and trained a linear layer on the 500 labels for 20 epochs. Test accuracy: 47.85%.

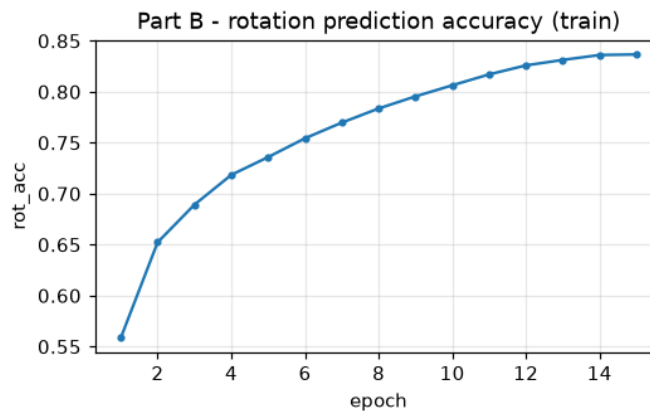


Figure 3: rotation-prediction accuracy during pretraining.

Part C - SimCLR-style contrastive learning

I used a random subset of 20,000 unlabeled images. Each image gets two views from the four demo augmentations; the encoder feeds an MLP projection head (512 to 512 to 128); the loss is NT-Xent with cosine similarity and temperature 0.2, with the other 2(N-1) views in the batch as negatives (batch 256, so 510 negatives). 20 epochs, AdamW 1e-3 with a cosine schedule. Then the projection head is removed, the encoder is frozen and a linear classifier is trained on the 500 labels for 20 epochs. Test accuracy: 44.62%.

Something I noticed in the log: the contrastive task became too easy. The fraction of anchors whose nearest other view is the true positive reached 97% by epoch 10 and 99% by epoch 20, and the loss flattened at 2.03. When two crops each cover 60-100% of the same image they share most of their pixels, so the network can solve the task from low-level cues like colour layout and never has to learn what the object is. This turned out to matter a lot, see the ablation below.

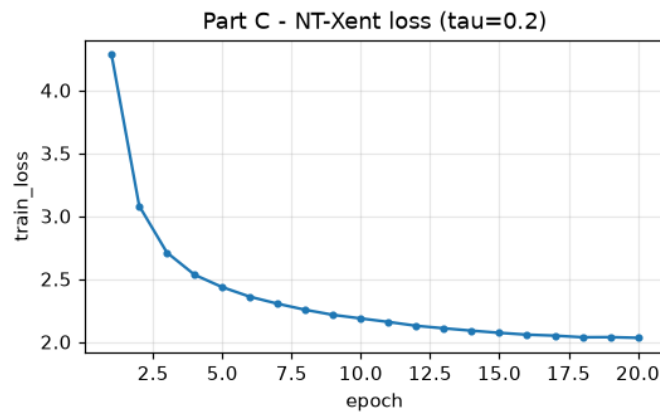


Figure 4: SimCLR NT-Xent loss (demo augmentations).

Results

Model	Test accuracy	Five-subset mean +- std	Top-5 NN precision
A - Supervised, 500 labels	43.95%	44.85 +- 1.13	39.66%
B - Rotation SSL + linear (100k images)	47.85%	47.06 +- 0.65	40.97%
C - SimCLR + linear (demo aug, 20k images)	44.62%	44.34 +- 0.69	39.89%
Extra: B' - rotation on the same 20k images	-	42.81 +- 0.28	-
Extra: C' - SimCLR, crop 0.2-1, no hue (ablation)	51.16%	50.36 +- 1.29	44.96%
Extra: random encoder + linear	28.59%	29.42 +- 1.54	-

The first column is the single run with SEED = 3215 (what the notebook reports). The second column repeats the 500-label part on five different stratified subsets (seeds 3215-3219): supervised training is redone from scratch for each subset, the linear probes are retrained, and the pretrained encoders stay the same. So it measures how much the labeled subset matters, not how much a different pretraining run would change things; each self-supervised method was pretrained only once.

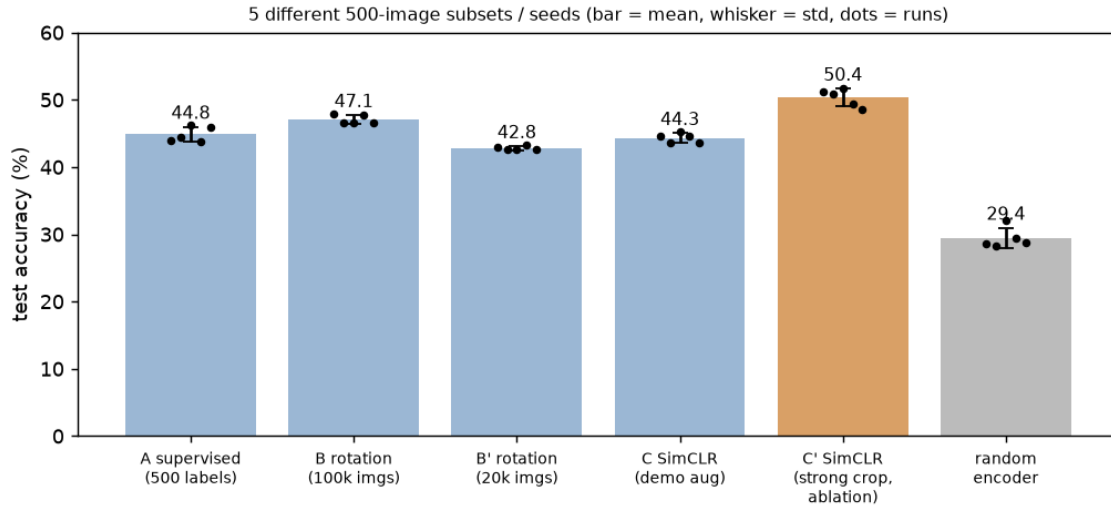


Figure 5: test accuracy over five different 500-image subsets.

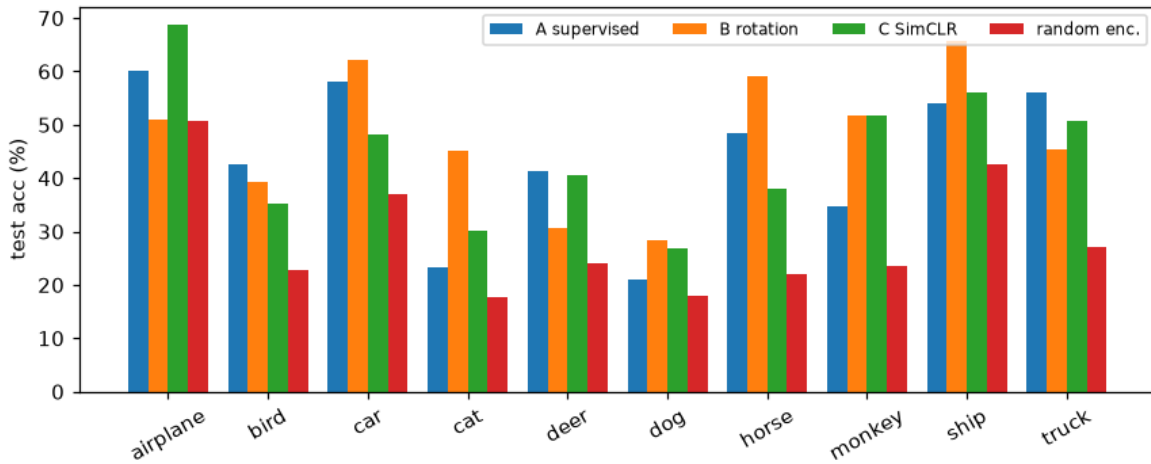
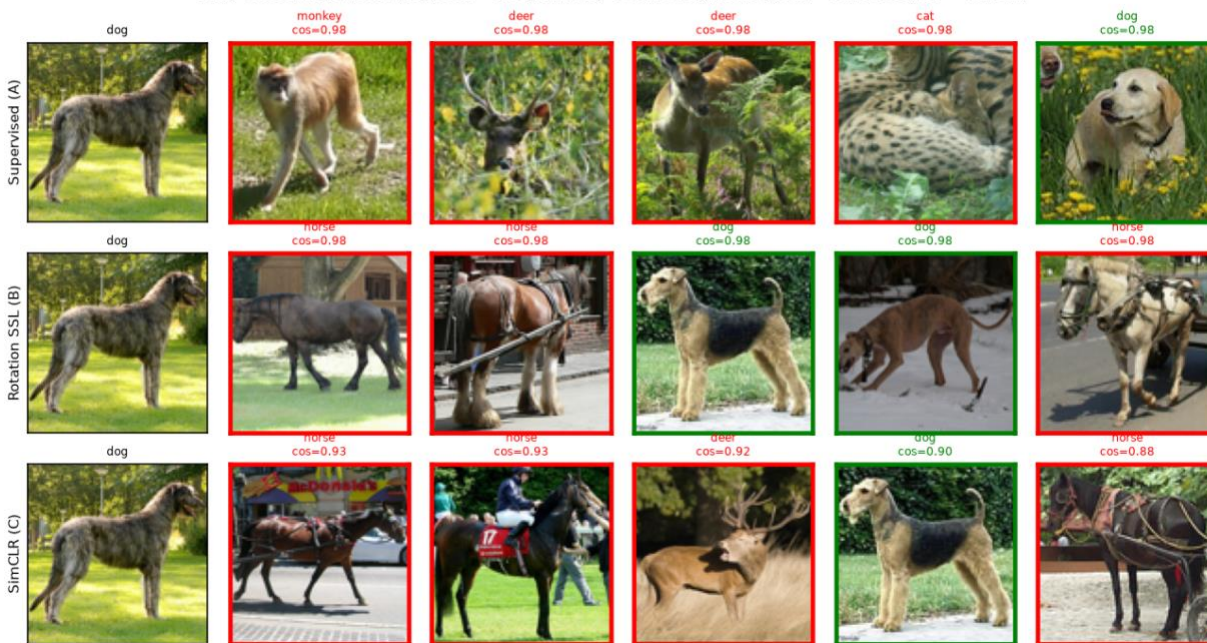


Figure 6: per-class test accuracy (main run).

Part D - Nearest-neighbour visualization

For each encoder I computed 512-dimensional embeddings of all 8,000 test images, L2-normalised them so that a dot product is cosine similarity, and retrieved the top-5 neighbours of each query among the other test images (the query itself is excluded). The same four queries are used for all three encoders: a dog (CLS_A), a car (CLS_B), an airplane and a deer. In the figures the green frame means the neighbour has the query's class and red means it does not; the number under each image is the cosine similarity.

Query 1: test image #6224 (dog). Left = query, then top-5 cosine neighbours (green = same class, red = different)



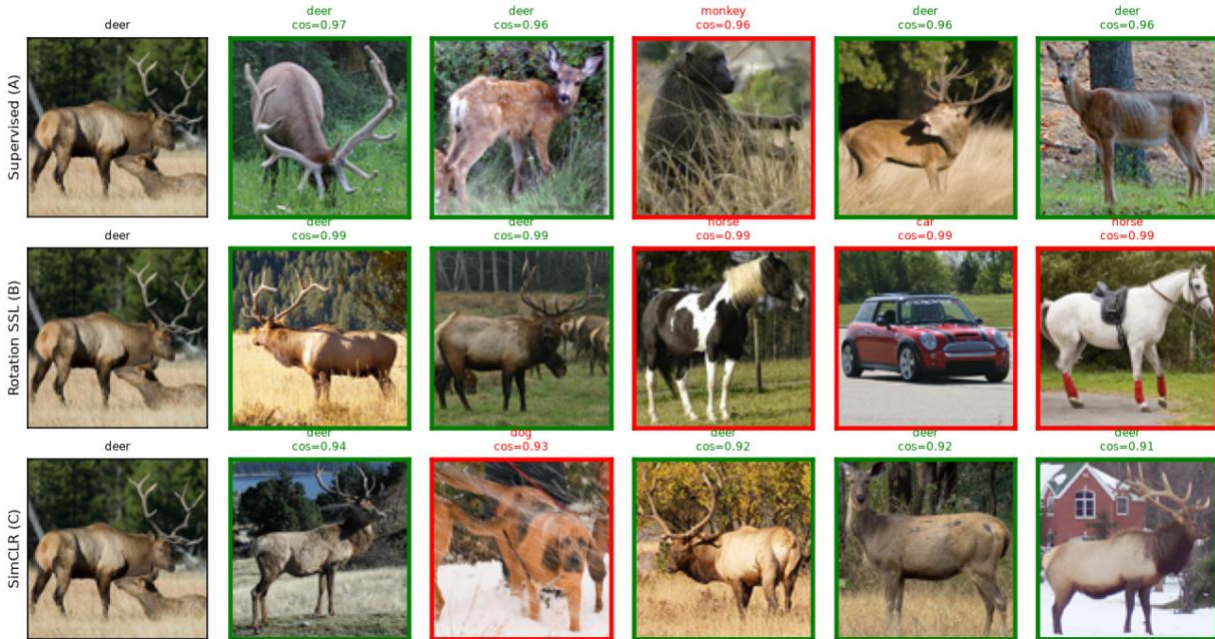
Query 2: test image #1842 (car). Left = query, then top-5 cosine neighbours (green = same class, red = different)



Query 3: test image #3770 (airplane). Left = query, then top-5 cosine neighbours (green = same class, red = different)



Query 4: test image #2209 (deer). Left = query, then top-5 cosine neighbours (green = same class, red = different)



Figures 7-10: query, then top-5 neighbours for the supervised (top), rotation (middle) and SimCLR (bottom) encoders.

Query (same-class neighbours out of 5)	Supervised	Rotation	SimCLR
dog	1	2	1
car	5	2	5
airplane	3	4	2
deer	4	2	4

Four queries cannot rank three models, so I also computed neighbour precision for all 8,000 test images as queries. Top-5 precision: supervised 39.66%, rotation 40.97%, SimCLR 39.89% (1-NN accuracy 42.40%, 44.74%, 43.22%). The number of queries (out of 8,000) where none of the five neighbours has the right class is 1,825 for supervised, 1,591 for rotation and 1,671 for SimCLR.

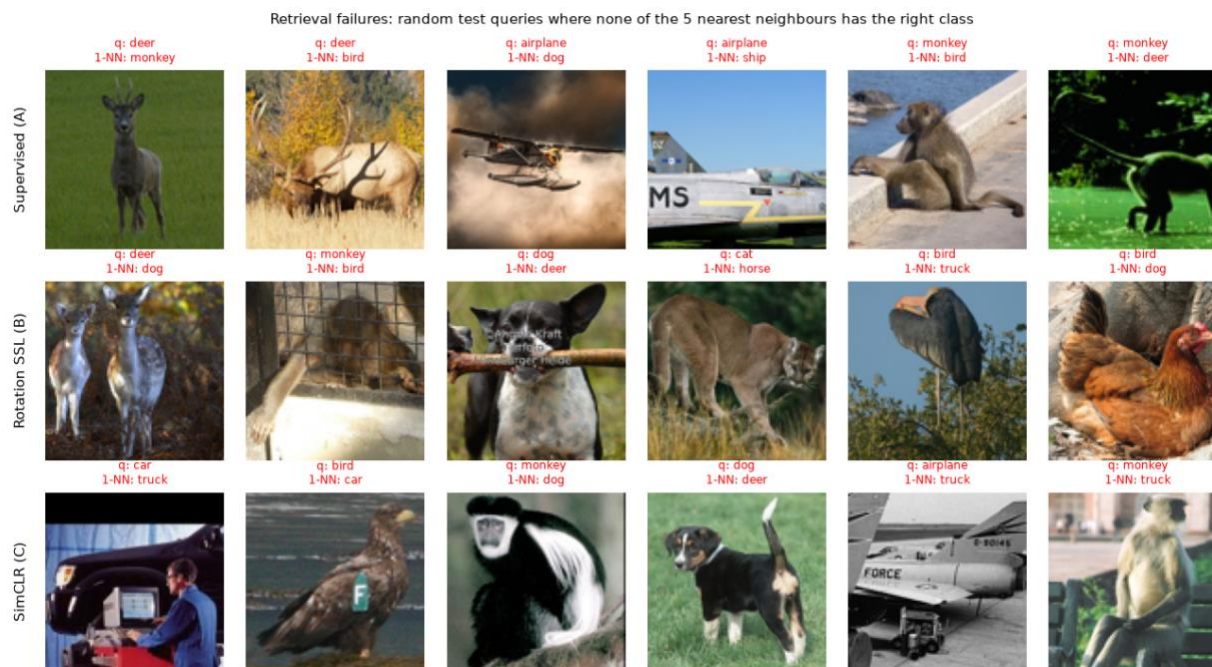


Figure 11: random test queries where all five neighbours are the wrong class.

Analysis

Which model performed best, and why

With the augmentations the assignment prescribes (the demo's), rotation SSL (B) performed best: 47.85% in the main run and 47.06 +- 0.65 over five subsets, which is about 2-3 points above both the supervised baseline (44.85 +- 1.13) and SimCLR (44.34 +- 0.69). The supervised model and SimCLR are not distinguishable, since their ranges overlap. All of them are far above a random frozen encoder (about 29%), so the pretraining did learn useful features. The neighbour retrieval agrees (rotation 41.0% > SimCLR 39.9% > supervised 39.7%), although those gaps are small.

Rotation worked well because the task cannot be solved from colour or position alone: the network has to recognise what the object is and which way is up, which is useful for ships (66% vs 43% for a random encoder), horses (59% vs 22%) and monkeys (52% vs 24%). It also saw 100,000 images, five times as many as SimCLR.

The ranking is not the whole story, though, and two extra experiments change how I read it:

- **Data-matched comparison.** When rotation is given the same 20,000 images as SimCLR it drops to 42.81 $\pm$ 0.28, below SimCLR with the demo augmentations (44.34 $\pm$ 0.69). So rotation's lead over demo-SimCLR comes from the extra data, not from a better objective.
- **Augmentation strength.** My first SimCLR run used a stronger crop (scale 0.2-1.0) and no hue jitter. That version gets 51.16% (50.36 $\pm$ 1.29 over five subsets) and beats every other model, including rotation with five times more data, by about 3 points. Its positive-pair accuracy stayed at 81.5% instead of 99%, so the task stayed hard enough to force object-level features. The demo's settings and mine differ in two things at once (crop scale and hue), so I cannot say which of the two matters, only that this combination does; I also only pretrained each version once.

Why the other models performed the way they did

- **Supervised (A).** 500 images are too few for a from-scratch ResNet-18: the training loss falls to 0.53 but the test accuracy stalls near 44%, and it is the weakest on cat (23%) and dog (21%), the classes that need more than 50 examples each. Its errors are mostly look-alike classes (car/truck 261, ship/truck 209).
- **SimCLR (C).** With the demo's augmentations the contrastive task was too easy (99% positive-pair accuracy), so the encoder stopped learning much beyond what the supervised model learns; it also saw only 20,000 images. With a harder augmentation it was the best model.
- **Rotation (B).** The pretext task was learned well, but rotation is partly solvable with cues that do not help classification (horizon lines, sky/ground position), and it says little about colour and texture, which separate bird, airplane and deer. Mid-network features would probably probe better than the final layer.

What changes could improve each model

- **Supervised:** Stronger augmentation (RandAugment, mixup/cutmix), label smoothing, more weight decay and early stopping on a slice of the 500; or, better, initialise from the rotation or SimCLR encoder and fine-tune.
- **Rotation:** Train longer (the rotation accuracy was still rising slowly at epoch 15), add a second pretext task (jigsaw, colorization), and probe intermediate layers.
- **SimCLR:** Make the contrastive task harder (crop scale 0.2-1.0 gave +6.5 points here, stronger colour jitter), use more negatives (larger batch or a memory queue), train 100+ epochs and use all 100,000 unlabeled images instead of 20,000.
- **Linear probes:** With only 500 labels the probe is a limiting factor for B and C; I used fixed hyperparameters and did not tune them on the test set. More labels or a tuned probe would help every model.

What the nearest-neighbour visualizations revealed

- **Mixed per-query picture.** No encoder wins on every query: supervised and SimCLR retrieve five cars, while rotation mixes in trucks (its main confusion); rotation is best on the airplane; the dog is hard for all three (one or two correct out of five).
- **What the mistakes look like.** For the floatplane query the wrong neighbours are boats on water, and for the SimCLR encoder also a truck and a horse. The encoders match the scene and background (water, grass, tarmac) as much as the object. Dog queries return horses, deer and monkeys: four-legged

animals on grass. So the embeddings capture coarse semantics (animal vs vehicle) but confuse look-alike classes and are easily pulled by the background.

- **Failure cases.** Failures happen with unusual poses, cluttered backgrounds and objects that fill the frame (a monkey on a bench whose nearest neighbour is a truck, a parked jet in a black-and-white photo). The pictures alone would not have shown which encoder is better; the all-test-set precision and the 0/5 counts did.
- **Cosine values.** Supervised and rotation embeddings have very high cosine similarities (0.94-0.99), SimCLR lower and more spread (0.87-0.94), which fits its loss pushing different images apart. The absolute values cannot be compared across encoders; only the ranking within one encoder is meaningful.

Overall, representation quality is better than random for all three, rotation is the best of the required models with the demo's augmentations, and the strongest result came from a harder SimCLR augmentation, which suggests the augmentation strength mattered more than the choice of method.

Limitations

- Each self-supervised method was pretrained once. The five-subset study varies the labeled subset and the supervised/probe training, not the pretraining seed.
- The pretraining runs are short (15-20 epochs) and the probes are not tuned, so absolute numbers are low compared with published STL-10 results.
- The B vs C comparison is not data-matched in the required setup (100k vs 20k images, as the assignment specifies); B' is the matched control.
- The stronger-crop ablation changes two settings at once, so it does not isolate the cause.
- Part A is the final-epoch accuracy (not the best epoch) and was noisy early in training.
- Parts A and B gave identical numbers in two separate full runs of the notebook, so the pipeline is deterministic on this machine, but other hardware (CUDA, CPU) will not reproduce the exact digits.

Files

HW6.ipynb (executed, with outputs and the analysis), README.md, METRICS.md (all numbers), RUN_LOG.txt (console output of every run), AI_USE.md, outputs/ (figures and results.json, robustness.json), checkpoints/ (the three encoders plus the 20k rotation encoder), ablation_strongcrop/ (the first full run), and scripts/ (notebook builder, robustness and plotting scripts, this document's builder). The encoder weights were produced by running HW6.ipynb top to bottom (jupyter nbconvert --to notebook --execute HW6.ipynb) and scripts/robustness.py.